*Name: YUVRAJ MAHILANGE*

*Course: Bsc. Computer Science (Hons.)*

*Topic: Assignments*

*Enrollment No.: GGV/24/05176*

Submitted to:

**Prof. Ratnesh Shrivastava**
Department of CSIT
GGV Bilaspur

QUESTION 1

Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

PROGRAM FOR THE QUESTION:

```
import java.io.*;
import java.util.*;

class Farmer {
   String name;
   String fruit;
   String city;
   int quantity;

   Farmer(String name, String fruit, String city, int quantity) {
      this.name = name;
      this.fruit = fruit;
      this.city = city;
      this.quantity = quantity;
   }
}

public class FarmerData {
   static void writeToFile(Farmer[] farmers, String filename) throws IOException {
      BufferedWriter bw = new BufferedWriter(new FileWriter(filename));
```

```
        String header = String.format("%-15s %-15s %-15s %-10s", "Name", "Fruit", "City", "Quantity");


 System.out.println("\n" + header);
bw.write(header);
        bw.newLine();
for (Farmer f : farmers) {
            String row = String.format("%-15s %-15s %-15s %-10d", f.name, f.fruit, f.city, f.quantity);
            System.out.println(row);
            bw.write(row);
            bw.newLine();
        }
        bw.close();
    }


    static void searchFarmer(Farmer[] farmers, String key) {
        boolean found = false; // Changed Boolean to boolean
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(key)) { // Fixed 'id' to 'if'
                System.out.println("\nFarmer Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Fruit: " + f.fruit);
                System.out.println("City: " + f.city);
                System.out.println("Quantity: " + f.quantity); // Fixed casing
                found = true;
                break; // Fixed quote to semicolon
            }
        }
```

```
        if (!found) {
            System.out.println("\nFarmer not Found.");
        }
    }
public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        Farmer[] farmers = new Farmer[10]; // Fixed 'Farmers' to 'Farmer'

        for (int i = 0; i < 10; i++) {
            System.out.println("\nEnter details for Farmer " + (i + 1));
            System.out.print("Name: ");
            String name = sc.nextLine();
            System.out.print("Fruit: ");
            String fruit = sc.nextLine();
            System.out.print("City: ");
            String city = sc.nextLine();
            System.out.print("Quantity: ");
            int quantity = sc.nextInt(); // Fixed data type
            sc.nextLine(); // Clear buffer

            farmers[i] = new Farmer(name, fruit, city, quantity);
        }
             System.out.println("Before Sort");
        writeToFile(farmers, "before_sort.txt"); // Fixed closing quote
        Arrays.sort(farmers, Comparator.comparingInt(f -> f.quantity));
             System.out.println("After Sort");
        writeToFile(farmers, "after_sort.txt");
```

```
        System.out.println("\nEnter farmer name to search:");
        String searchName = sc.nextLine();
        searchFarmer(farmers, searchName); // Fixed 'darmers' typo
        sc.close();
            }
}
```

OUTPUT:

```
c:\myJavaFile>javac FarmerData.java
C:\myJavaFile>java -cp . FarmerData

Enter details for Farmer 1
Name: F1
Fruit: APPLE
City: BSP
Quantity: 50

Enter details for Farmer 2
Name: F2
Fruit: BANANA
City: BSP
Quantity: 340

Enter details for Farmer 3
Name: F3
Fruit: ORANGE
```

City: RAIPUR

Quantity: 90

Enter details for Farmer 4

Name: F4

Fruit: LITCHI

City: BIHAR

Quantity: 300

Enter details for Farmer 5

Name: F5

Fruit: LITCHI

City: RAIGARH

Quantity: 200

Enter details for Farmer 6

Name: F6

Fruit: BANANA

City: EUROPE

Quantity: 140

Enter details for Farmer 7

Name: F7

Fruit: MANGO

City: BOTSWANA

Quantity: 250

Enter details for Farmer 8

Name: F8

Fruit: STRAWBERRY

City: FRANCE

Quantity: 450

Enter details for Farmer 9

Name: F9

Fruit: MELON

City: LADAKH

Quantity: 400

Enter details for Farmer 10

Name: F10

Fruit: MANGO

City: AFRICA

Quantity: 200

| Name | Fruit | City | Quantity |
|---|---|---|---|
| F1 | APPLE | BSP | 50 |
| F2 | BANANA | BSP | 340 |
| F3 | ORANGE | RAIPUR | 90 |
| F4 | LITCHI | BIHAR | 300 |
| F5 | LITCHI | RAIGARH | 200 |
| F6 | BANANA | EUROPE | 140 |
| F7 | MANGO | BOTSWANA | 250 |
| F8 | STRAWBERRY | FRANCE | 450 |

| | | | |
|---|---|---|---|
| F9 | MELON | LADAKH | 400 |
| F10 | MANGO | AFRICA | 200 |

| Name | Fruit | City | Quantity |
|---|---|---|---|
| F1 | APPLE | BSP | 50 |
| F3 | ORANGE | RAIPUR | 90 |
| F6 | BANANA | EUROPE | 140 |
| F5 | LITCHI | RAIGARH | 200 |
| F10 | MANGO | AFRICA | 200 |
| F7 | MANGO | BOTSWANA | 250 |
| F4 | LITCHI | BIHAR | 300 |
| F2 | BANANA | BSP | 340 |
| F9 | MELON | LADAKH | 400 |
| F8 | STRAWBERRY | FRANCE | 450 |

Enter farmer name to search:

F2

Farmer Found:

Name: F2

Fruit: BANANA

City: BSP

Quantity: 340

QUESTION 2

Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

PROGRAM FOR THE CODE:

```
import java.util.*;


// Class to store fruit data
class Fruit {
   String name;
   String quality;
   int quantity;

   Fruit(String name, String quality, int quantity) {
      this.name = name;
      this.quality = quality;
      this.quantity = quantity;
   }
}

public class FruitData {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Fixed fruits
      String[] fruitNames = {"Mango", "Apple", "Papaya"};
      Fruit[] fruits = new Fruit[3];
```

```
// Input
for (int i = 0; i < 3; i++) {
    System.out.println("\nEnter details for " + fruitNames[i]);

    System.out.print("Quality (good/average/bad): ");
    String quality = sc.nextLine();

    System.out.print("Quantity: ");
    int quantity = sc.nextInt();
    sc.nextLine(); // clear buffer

    // If quantity > 1000, store half
    if (quantity > 1000) {
        quantity = quantity / 2;
    }

    fruits[i] = new Fruit(fruitNames[i], quality, quantity);
}

// Display stored data
System.out.println("\nStored Data:");
for (Fruit f : fruits) {
    System.out.println(f.name + " | " + f.quality + " | " + f.quantity);
}

// Search
```

```
System.out.println("\nEnter fruit name to search:");
String searchFruit = sc.nextLine();

System.out.println("Enter quality to search:");
String searchQuality = sc.nextLine();

boolean found = false;

for (Fruit f : fruits) {
    if (f.name.equalsIgnoreCase(searchFruit) &&
        f.quality.equalsIgnoreCase(searchQuality)) {

        System.out.println("\nFruit Found:");
        System.out.println("Name: " + f.name);
        System.out.println("Quality: " + f.quality);
        System.out.println("Quantity: " + f.quantity);

        found = true;
        break;
    }
}

if (!found) {
    System.out.println("\nFruit not found.");
}

sc.close();
```

```
    }
}
```

OUTPUT:

```
C:\myJavaFile>java -cp . FruitData

Enter details for Mango
Quality (good/average/bad): good
Quantity: 900

Enter details for Apple
Quality (good/average/bad): bad
Quantity: 300

Enter details for Papaya
Quality (good/average/bad): Average
Quantity: 1200

Stored Data:
Mango | good | 900
Apple | bad | 300
Papaya | Average | 600

Enter fruit name to search:
Apple
Enter quality to search:
bad
```

Fruit Found:

Name: Apple

Quality: bad

Quantity: 300

QUESTION 3

There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

PROGRAM FOR THE QUESTION:

```
import java.util.Scanner;

// Node class for tree structure
class OrchardNode {
    String orchardName;
    String fruit;
    String quality;
    OrchardNode left, right;

    OrchardNode(String orchardName, String fruit, String quality) {
        this.orchardName = orchardName;
        this.fruit = fruit;
        this.quality = quality;
        left = right = null;
    }
}

public class OrchardTree {

    // Insert node level-wise manually for fixed 4 orchards
    static OrchardNode createTree() {
        OrchardNode root = new OrchardNode("A", "Apple", "Good");
```

```
    root.left = new OrchardNode("B", "Litchi", "Good");
    root.right = new OrchardNode("C", "Mango", "Good");
    root.left.left = new OrchardNode("D", "Apple", "Bad");

    return root;
  }

  // Print tree in simple tree-view
  static void printTree(OrchardNode node, int level) {
    if (node == null)
      return;

    for (int i = 0; i < level; i++)
      System.out.print("   ");

    System.out.println(node.orchardName + " -> " +
        node.fruit + " (" + node.quality + ")");

    printTree(node.left, level + 1);
    printTree(node.right, level + 1);
  }

  // Search fruit in tree
  static void searchFruit(OrchardNode node, String key) {
    if (node == null)
      return;

    if (node.fruit.equalsIgnoreCase(key)) {
      System.out.println("Found in Orchard " +
          node.orchardName + " | Quality: " + node.quality);
    }

    searchFruit(node.left, key);
    searchFruit(node.right, key);
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    OrchardNode root = createTree();
```

```
        System.out.println("Available Orchards in Tree View:");
        printTree(root, 0);

        System.out.print("\nEnter fruit name to search: ");
        String fruitName = sc.nextLine();

        System.out.println("\nSearch Result:");
        searchFruit(root, fruitName);

        System.out.println("\nTree Structure:");
        printTree(root, 0);

        sc.close();
    }
}
```

OUTPUT:

```
c:\myJavaFile>javac OrchardTree.java

C:\myJavaFile>java -cp . OrchardTree

Available Orchards in Tree View:
A -> Apple (Good)
  B -> Litchi (Good)
    D -> Apple (Bad)
  C -> Mango (Good)

Enter fruit name to search: Apple

Search Result:
Found in Orchard A | Quality: Good
Found in Orchard D | Quality: Bad

Tree Structure:
A -> Apple (Good)
  B -> Litchi (Good)
    D -> Apple (Bad)
  C -> Mango (Good)
```

QUESTION 4

Take a map provided as a graph & display all the subtrees of connected points.

PROGRAM FOR THE QUESTION:

```
import java.util.*;

// Graph using adjacency list
public class GraphSubtrees {

    static int vertices = 7; // Total points
    static ArrayList<Integer>[] graph = (ArrayList<Integer>[]) new ArrayList[vertices];
    static boolean[] visited = new boolean[vertices];

    // Add connection between two points
    static void addEdge(int u, int v) {
        graph[u].add(v);
        graph[v].add(u);
    }

    // DFS to print one connected subtree
    static void dfs(int node) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int next : graph[node]) {
            if (!visited[next]) {
                dfs(next);
```

```
        }
    }
}

public static void main(String[] args) {

    // Initialize graph
    for (int i = 0; i < vertices; i++) {
        graph[i] = new ArrayList<>();
    }

    // Sample map connections
    addEdge(0, 1);
    addEdge(1, 2);

    addEdge(3, 4);

    addEdge(5, 6);

    System.out.println("All Subtrees of Connected Points:");

    // Find all connected components
    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            System.out.print("Subtree: ");
            dfs(i);
            System.out.println();
```

```
            }
        }
    }
}
```

OUTPUT:

```
C:\myJavaFile>javac GraphSubtrees.java
Note: GraphSubtrees.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

C:\myJavaFile>java -cp . GraphSubtrees
All Subtrees of Connected Points:
Subtree: 0 1 2
Subtree: 3 4
Subtree: 5 6
```

QUESTION 5

Enter entry point & goal point & produce the matching subtree of the graph.

PROGRAM FOR THE QUESTION:

```
import java.util.*;

// Find path (matching subtree) between entry point and goal point
public class GraphPath {

    static int vertices = 7;
    static ArrayList<Integer>[] graph = new ArrayList[vertices];
    static boolean[] visited = new boolean[vertices];
    static ArrayList<Integer> path = new ArrayList<>();

    // Add edge between points
    static void addEdge(int u, int v) {
        graph[u].add(v);
```

```
        graph[v].add(u);
    }

    // DFS to find path
    static boolean findPath(int current, int goal) {
        visited[current] = true;
        path.add(current);

        if (current == goal)
            return true;

        for (int next : graph[current]) {
            if (!visited[next]) {
                if (findPath(next, goal))
                    return true;
            }
        }

        // Backtrack if wrong path
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Initialize graph
        for (int i = 0; i < vertices; i++) {
            graph[i] = new ArrayList<>();
        }

        // Sample graph
        addEdge(0, 1);
        addEdge(1, 2);
        addEdge(1, 3);
        addEdge(3, 4);
        addEdge(4, 5);
        addEdge(5, 6);

        System.out.print("Enter Entry Point: ");
```

```
        int start = sc.nextInt();

        System.out.print("Enter Goal Point: ");
        int goal = sc.nextInt();

        if (findPath(start, goal)) {
            System.out.println("\nMatching Subtree / Path:");
            for (int node : path) {
                System.out.print(node + " ");
            }
        } else {
            System.out.println("\nNo Path Found.");
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\myJavFile>javac GraphPath.java
Note: GraphPath.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

C:\myJavaFile>java -cp . GraphPath
Enter Entry Point: 0
Enter Goal Point: 6

Matching Subtree / Path:
0    1 3 4 5 6
```

QUESTION 6

Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

PROGRAM FOR THE QUESTION:

```
import java.util.*;

// Shortest path using Dijkstra Algorithm
public class ShortestPathGraph {
```

```
static class Edge {
    int to, weight;

    Edge(int to, int weight) {
        this.to = to;
        this.weight = weight;
    }
}

static int vertices = 6;
static ArrayList<Edge>[] graph = new ArrayList[vertices];

// Add weighted edge
static void addEdge(int u, int v, int w) {
    graph[u].add(new Edge(v, w));
    graph[v].add(new Edge(u, w));
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    for (int i = 0; i < vertices; i++) {
        graph[i] = new ArrayList<>();
    }

    // Sample weighted graph
    addEdge(0, 1, 4);
    addEdge(0, 2, 2);
    addEdge(1, 3, 5);
    addEdge(2, 1, 1);
    addEdge(2, 4, 8);
    addEdge(3, 5, 3);
    addEdge(4, 5, 2);

    System.out.print("Enter Start Node: ");
    int start = sc.nextInt();

    System.out.print("Enter End Node: ");
    int end = sc.nextInt();
```

```
int[] dist = new int[vertices];
int[] parent = new int[vertices];
boolean[] visited = new boolean[vertices];

Arrays.fill(dist, Integer.MAX_VALUE);
Arrays.fill(parent, -1);

dist[start] = 0;

// Dijkstra Algorithm
for (int i = 0; i < vertices - 1; i++) {
    int u = -1;
    int min = Integer.MAX_VALUE;

    for (int j = 0; j < vertices; j++) {
        if (!visited[j] && dist[j] < min) {
            min = dist[j];
            u = j;
        }
    }

    if (u == -1)
        break;

    visited[u] = true;

    for (Edge e : graph[u]) {
        if (!visited[e.to] && dist[u] + e.weight < dist[e.to]) {
            dist[e.to] = dist[u] + e.weight;
            parent[e.to] = u;
        }
    }
}

// Print shortest path
if (dist[end] == Integer.MAX_VALUE) {
    System.out.println("No Path Found.");
} else {
    ArrayList<Integer> path = new ArrayList<>();
    for (int v = end; v != -1; v = parent[v]) {
```

```
            path.add(v);
        }

        Collections.reverse(path);

        System.out.println("\nShortest Path:");
        for (int node : path) {
            System.out.print(node + " ");
        }

        System.out.println("\nTotal Cost: " + dist[end]);
    }

    sc.close();
  }
}
```

OUTPUT:

```
c:\myJavaFile>javac ShortestPathGraph.java
C:\myJavaFile>java -cp . ShortestPathGraph
Enter Start Node: 0
Enter End Node: 5

Shortest Path:
0 2 1 3 5
Total Cost: 11
```

QUESTION 7

Write a program to demonstrate the use of all keywords made for exception handling.

PROGRAM FOR THE QUESTION:

```
import java.util.*;

public class ExceptionDemo {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
```

```
            System.out.print("Enter first number: ");
            int a = sc.nextInt();

            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            int result = divide(a, b);   // may throw exception
            System.out.println("Result = " + result);
        }

        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        catch (InputMismatchException e) {
            System.out.println("Invalid input. Enter integers only.");
        }

        catch (Exception e) {
            System.out.println("General Exception: " + e);
        }

        finally {
            System.out.println("Finally block executed.");
            sc.close();
        }
    }

    // Method using throws keyword
    static int divide(int x, int y) throws ArithmeticException {
        if (y == 0) {
            throw new ArithmeticException("Divide by zero");
        }
        return x / y;
    }
}
```

OUTPUT:

```
c:\myJavaFile>javac ExceptionDemo.java
c:\myJavaFile>java -cp . ExceptionDemo
```

```
Enter first number: 45
Enter second number: 63
Result = 0
Finally block executed.

c:\myJavaFile>java -cp . ExceptionDemo
Enter first number: 152
Enter second number: 0
Cannot divide by zero.
Finally block executed.

c:\myJavaFile>java -cp . ExceptionDemo
Enter first number: abc
Invalid input. Enter integers only.
Finally block executed.
```

QUESTION 8

Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

PROGRAM FOR THE QUESTION:

```
import java.util.*;

// User-defined exception
class NoMoneyException extends Exception {
   NoMoneyException(String message) {
      super(message);
   }
}

public class BankWithdraw {

   static void withdraw(double balance, double amount) throws NoMoneyException {

      if (amount > balance) {
         throw new NoMoneyException("Insufficient Balance.");
      }

      balance = balance - amount;
```

```
        System.out.println("Withdrawal Successful.");
        System.out.println("Remaining Balance: " + balance);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter Account Balance: ");
            double balance = sc.nextDouble();

            System.out.print("Enter Withdrawal Amount: ");
            double amount = sc.nextDouble();

            withdraw(balance, amount);
        }

        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        catch (Exception e) {
            System.out.println("Invalid Input.");
        }

        finally {
            sc.close();
        }
    }
}
```

OUTPUT:

```
C:\myJavaFile>javac BankWithdraw.java

C:\myJavaFile>java -cp . BankWithdraw
Enter Account Balance: 5000
Enter Withdrawal Amount: 1200
Withdrawal Successful.
```

```
Remaining Balance: 3800.0

C:\myJavaFile>java -cp . BankWithdraw
Enter Account Balance: 6000
Enter Withdrawal Amount: 12000
Exception: Insufficient Balance.

C:\myJavaFile>java -cp . BankWithdraw
Enter Account Balance: ab
Invalid Input.
```

QUESTION 9

Write a program which enables to use a platform by a train one at a time.

PROGRAM FOR THE QUESTION:

```
class Platform {

   // synchronized allows only one train at a time
   synchronized void usePlatform(String trainName) {

      System.out.println(trainName + " entered platform.");

      try {
         Thread.sleep(2000); // train stays for 2 seconds
      } catch (Exception e) {
      }

      System.out.println(trainName + " left platform.");
   }
}

// Train thread
class Train extends Thread {

   Platform p;
   String name;

   Train(Platform p, String name) {
      this.p = p;
      this.name = name;
```

```
    }

    public void run() {
        p.usePlatform(name);
    }
}

public class RailwayPlatform {

    public static void main(String[] args) {

        Platform p = new Platform();

        Train t1 = new Train(p, "Train A");
        Train t2 = new Train(p, "Train B");
        Train t3 = new Train(p, "Train C");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT:

```
C:\myJavaFile>javac RailwayPlatform.java

C:\myJavaFile>java -cp . RailwayPlatform
Train B entered platform.
Train B left platform.
Train A entered platform.
Train A left platform.
Train C entered platform.
Train C left platform.

C:\myJavaFile>java -cp . RailwayPlatform
Train C entered platform.
Train C left platform.
Train B entered platform.
Train B left platform.
```

Train A entered platform.
Train A left platform.

QUESTION 10

Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

PROGRAM FOR THE QUESTION:

```
class Restaurant {

    boolean kitchenFree = true;
    boolean chefFree = true;

    // Waiter places order
    synchronized void placeOrder(String orderName) {

        if (kitchenFree) {
            kitchenFree = false;

            System.out.println("Waiter accepted order: " + orderName);
            System.out.println("Kitchen preparing " + orderName);

            try {
                Thread.sleep(2000);
            } catch (Exception e) {
            }

            kitchenFree = true;
        } else {
            System.out.println("Kitchen busy. Order rejected: " + orderName);
        }
    }

    // Chef delivers order
    synchronized void deliverOrder(String orderName) {

        if (chefFree) {
            chefFree = false;
```

```
            System.out.println("Chef delivered: " + orderName);

            try {
                Thread.sleep(1000);
            } catch (Exception e) {
            }

            chefFree = true;
        } else {
            System.out.println("Chef busy.");
        }
    }
}

// Waiter thread
class Waiter extends Thread {

    Restaurant r;
    String order;

    Waiter(Restaurant r, String order) {
        this.r = r;
        this.order = order;
    }

    public void run() {
        r.placeOrder(order);
    }
}

// Chef thread
class Chef extends Thread {

    Restaurant r;
    String order;

    Chef(Restaurant r, String order) {
        this.r = r;
        this.order = order;
```

```
    }

    public void run() {
        r.deliverOrder(order);
    }
}

public class RestaurantSystem {

    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        Waiter w1 = new Waiter(r, "Pizza");
        Waiter w2 = new Waiter(r, "Burger");

        Chef c1 = new Chef(r, "Pizza");
        Chef c2 = new Chef(r, "Burger");

        w1.start();
        w2.start();

        c1.start();
        c2.start();
    }
}
```

OUTPUT:

```
C:\myJavaFile>javac RestaurantSystem.java
C:\myJavaFile>java -cp . RestaurantSystem
Waiter accepted order: Burger
Kitchen preparing Burger
Chef delivered: Burger
Chef delivered: Pizza
Waiter accepted order: Pizza
Kitchen preparing Pizza
```